

Understanding Joins-53

Introduction

In relational databases, data is often split across multiple tables to avoid redundancy. To analyse or fetch meaningful information, you need to combine these tables. **Joins** in SQL allow you to retrieve data from two or more tables based on a related column, usually a **foreign key–primary key relationship**.

In MySQL Workbench, joins are essential when working with normalized databases where employee details, departments, orders, or customers are stored in separate tables.

Types of Joins in MySQL

1. INNER JOIN

Returns only the rows where there is a match in both tables unmatched rows are excluded.

- Syntax:

```
select salary.Firstname, salary.Salary, dept.Deptid from salary inner join Dept on salary.Deptid =Dept.Deptid;
```

here we are using inner join to get data from both salary and dept table

The screenshot displays the MySQL Workbench interface. The 'Query' tab is active, showing a query that performs an inner join between the 'salary' and 'dept' tables. The query is as follows:

```
116 (2354,'Biki',22500,'HR', 234, 154,956);
117
118 select salary.Firstname, salary.Salary, dept.Deptid from salary inner join Dept on salary.Deptid =Dept.Deptid;
119
120 select salary.Lastname, dept.Firstname from salary left join Dept on salary.Deptid =Dept.Deptid;
121
122 select salary.Firstname, dept.Deptid from salary right join dept on salary.Deptid = Dept.Deptid;
123
124 select salary.Firstname, dept.Deptid from salary full join dept on salary.Deptid = dept.Deptid;
```

The 'Result Grid' shows the output of the first query, displaying columns 'Firstname', 'Salary', and 'Deptid'. The results are:

Firstname	Salary	Deptid
Sheshadri	19500	201
Nushina	21000	201
Athulya	26580	209
Tammashte	22500	209
Radhu	21500	219

The 'Action Output' tab at the bottom shows the execution log, including the following entries:

- 55 14:11:41 delete from emp where Firstname='Nushina'
- 56 14:14:09 delete from emp where employeeid=2356
- 57 15:09:22 alter table emp comment = 'this is the old employee table'
- 58 15:10:25 show databases
- 59 15:11:15 use salary
- 60 15:11:27 create table Dept (Employeeid INT not null, Firstname VARCHAR(20), Salary INT, Dept varchar(20)...
- 61 15:11:31 create table Dept (Employeeid INT not null, Firstname VARCHAR(20), Salary INT, Dept varchar(20)...
- 62 15:12:50 insert into dept (Employeeid, Firstname, Salary, Dept, Deptid, Tax, PP) VALUES (2345,'Nushina', 210...
- 63 15:14:46 select salary.Firstname, salary.Salary, dept.Deptid from salary inner join Dept on salary.Deptid =Dept...

- Example: Shows employees and their department names, but only for employees who belong to an existing department.

2. LEFT JOIN (LEFT OUTER JOIN)

Returns all rows from the **left table**, and the matched rows from the right table. If no match exists, right table columns show as NULL.

- Syntax:

```
select salary.Lastname, dept.Firstname from salary left join Dept on salary.Deptid =Dept.Deptid;
```

Here we are using the left join to get data from left side

The screenshot shows the SQL Developer interface with a query window and a results grid. The query window displays the following SQL code:

```
116 (2354, 'Biki', 22500, 'HR', 234, 154, 956);
117
118 select salary.Firstname, salary.Salary, dept.Deptid from salary inner join Dept on salary.Deptid =Dept.Deptid;
119 select salary.Lastname, dept.Firstname from salary left join Dept on salary.Deptid =Dept.Deptid;
120 select salary.Firstname, dept.Dept from salary right join dept on salary.Deptid = Dept.Deptid;
121 select salary.Firstname, dept.Dept from salary full join dept on salary.Deptid = dept.Deptid;
```

The results grid shows the output of the query. The columns are Lastname and Firstname. The data is as follows:

Lastname	Firstname
Noushadh	Sheshadri
Noushadh	Nushima
Saravana	Athulya
Saravana	Tannishtha
Radhavan	Madhu

The Action Output window shows the execution of the query, indicating that 17 rows were returned.

- Example: Shows all employees. If an employee is not assigned a department, the Dept column will be NULL.

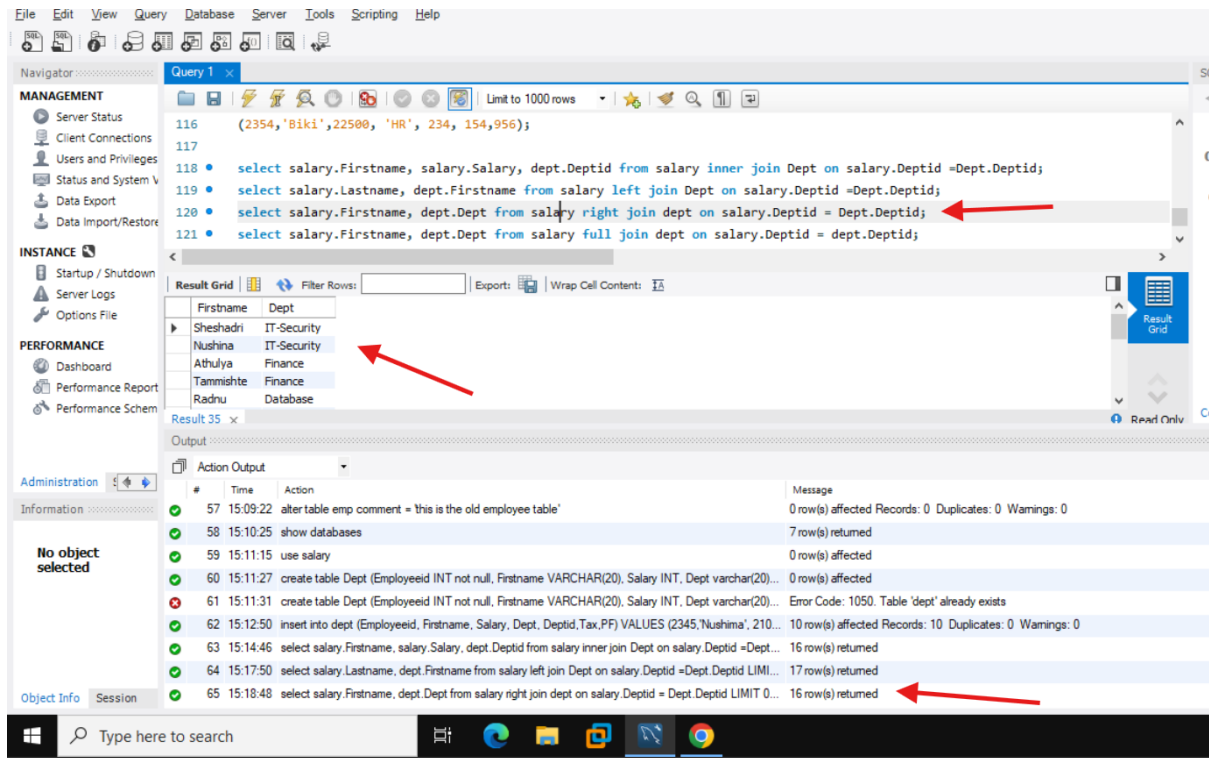
3. RIGHT JOIN (RIGHT OUTER JOIN)

Opposite of LEFT JOIN. Returns all rows from the **right table**, and the matched rows from the left table. If no match exists, left table columns show as NULL.

- Syntax:

```
select salary.Firstname, dept.Dept from salary right join dept on salary.Deptid = Dept.Deptid;
```

here we are using right join



- Example: Shows all departments, even if no employee belongs to a department.

4. FULL OUTER JOIN

Returns all rows from both tables, whether a match exists or not. **MySQL does not support FULL OUTER JOIN directly.** You simulate it using UNION of LEFT JOIN and RIGHT JOIN:

```

SELECT s.Firstname, s.Salary, d.Dept
FROM salary s
LEFT JOIN dept d ON s.Deptid = d.Deptid
UNION
SELECT s.Firstname, s.Salary, d.Dept
FROM salary s
RIGHT JOIN dept d ON s.Deptid = d.Deptid;

```

Workbench sir's example

```

select salary.Firstname, dept.Dept from salary full join dept on salary.Deptid =
dept.Deptid;

```

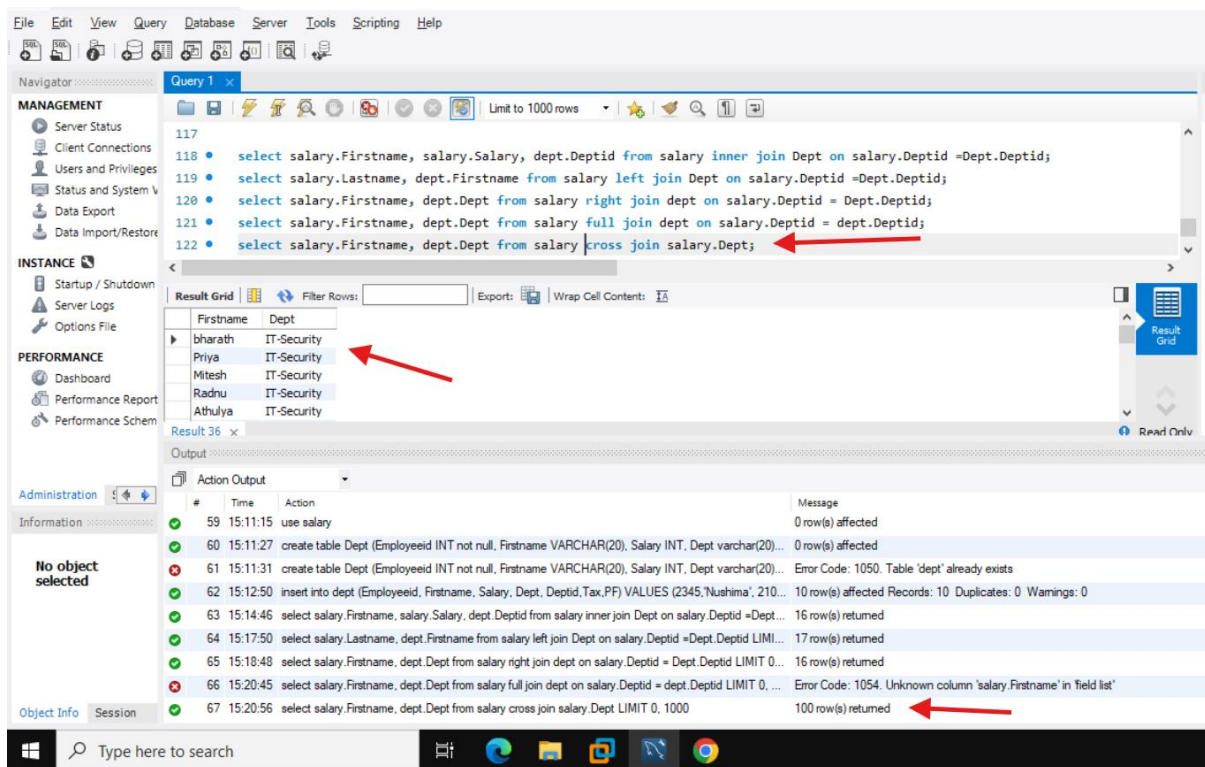
- Example: Shows all employees and all departments, with NULLs where matches are missing.

5. CROSS JOIN

Produces the Cartesian product of two tables. Every row from the first table combines with every row from the second.

- Syntax:

`select salary.Firstname, dept.Dept from salary cross join salary.Dept;`



- Example: If salary has 5 rows and dept has 3 rows, result will be 15 rows.

Important's

- Avoid redundancy by keeping related data in separate tables.
- Retrieve meaningful reports, such as employee with department, sales with customers, or orders with products.
- Improve database normalization and integrity.
- Allow complex queries to analyze relationships across multiple entities.